

Accurate and fast Dynamic Time Warping approximation using upper bounds

Bilel Ben Ali, Youssef Masmoudi, and Souhail Dhoubi

Abstract—The purpose of Dynamic Time Warping (DTW) is to determine the shortest warp path, corresponding to the optimal alignment between two sequences. It is one of the most used methods for time series distance measure. DTW was introduced to the community as a Data Mining utility for various tasks for time series problems such as classification and clustering. Many variants of DTW aim to accelerate the calculation of this distance and others order to overcome the weakness points of DTW as the singularity problem. We propose a new approach called Dynamic Warping Window (DWW) for speeding up DTW algorithm based on upper bound. It gives an accurate approximation of DTW in linear time. It accelerates the calculation of DTW distance. Results show that the new approach provides a good compromise between accuracy of DTW approximation and the non-degradation of KNN classification results.

Keywords—Data Mining; Dynamic Time Warping; Dynamic Warping Window; KNN classification; Time series; Metrics.

I. INTRODUCTION

THE problem of comparing time series is typically addressed by the algorithm of Dynamic Time Warping (DTW). This algorithm can be defined as a measure of distance between two series with temporal inhomogeneous changes (expansions or contractions). The distance given by the algorithm DTW has been used in many fields of applications such as speech recognition [1][2], gesture recognition [3][4] or computer animation [5]. The DTW algorithm proceeds by constructing a matrix of cumulative distances from which is extracted a least cost warping path. In its basic version, the DTW algorithm has two problems: First, the construction phase of the cumulative distance matrix involves a quadratic complexity in the length of sequences to be compared, and secondly, the local behavior of this type of algorithm tends to generate warping paths with sharp discontinuities. To address these limitations, we rely on an extension based on the use of the upper bounding functions to build a window around the diagonal path. The size of this warping window varies dynamically.

The rest of this paper is organized as follows: in the first section we will present the classic dynamic time warping algorithm. Then we talk about Derivative Dynamic Time Warping (DDTW) a variant proposed by [6] that try to overcome some disadvantages of DTW in the alignment of time series. The third section will be about the Dynamic Warping Window

a new variant of DTW that accelerates its CPU time while giving an accurate approximation of DTW distance. The fourth section presents experimental results and interpretations. In this section we demonstrate the efficiency of DWW over DDTW and DTW in terms of classification error rates and CPU time. Finally we finish by a conclusion and future works.

II. DYNAMIC TIME WARPING

In this section, Dynamic Time Warping (DTW) algorithm is presented. It provides effectively all possible warping paths and gives the minimal warping cost between two time series. Then different optimization techniques of DTW algorithm are presented.

A. Classic Dynamic Time Warping algorithm

This method is based on the technical management of deformations on the time axis. It was first used to match signals for speech recognition [7]. (Berndt and Clifford) [8] proposed to use this technique to measure the similarity of time series in the field of data mining. Other recent studies have also used this measure of similarity in data mining [9]. The dynamic time warping (DTW) algorithm is usually based on the idea of dynamic programming (DP) [10] to compare two time series with different size. Simply, it is to build an adjacency matrix to find the shortest warping path.

By considering $Q = \{q_1, q_2, \dots, q_m\}$ the first time series sequence and $C = \{c_1, c_2, \dots, c_n\}$ the second one. The algorithm begin by creating a matrix D of size $(m * n)$ with:

- m : the number of vectors in the first sequence.
- n : the number of vectors in the second sequence.

Each entry (i, j) of this matrix is associated with the squared euclidean distance $d(q_i, c_j)$. Every element of this new matrix is calculated like the sum between the local distance $d(q_i, c_j)$ and the minimum of the total distances of the immediately surrounding elements. To find the best distance $DTW(Q, C)$ between the first sequence C and the second sequence Q , just look for the "path" in the matrix D so as to minimize the sum of local distances encountered from the start point $(1, 1)$, corresponding to the beginning of the two sequences, to the end point (m, n) corresponding to the end of the two sequences.

It can be shown that the optimum distance is obtained by calculating, for each entry (i, j) the accumulated distance $D(i, j)$ corresponding to the optimum distance that is obtained by comparing the two sub-sequences corresponding to the first i points of Q and the j first elements of the second time series C . Under the principle of dynamic programming, we

Manuscript received April 21, 2014.

Bilel Ben Ali, LOGIQ Research Unit, University of Sfax, Tunisia; e-mail: bilel_benali@yahoo.fr).

Youssef Masmoudi, High School of Commerce, University of Sfax, Tunisia; e-mail: youssef.masmoudi@gmail.com).

Souhail Dhoubi, AL-Baha University, Arabie Saoudite; e-mail: souhail.dhoubi@gmail.com).

can easily show that this distance can be calculated using the following recurrence (equation (1)):

$$D(i, j) = \begin{cases} d(1, 1), & \text{if } i = j = 1; \\ d(i, j) + \min \{D(i-1, j), D(i, j-1), D(i-1, j-1)\}, & \\ \text{otherwise;} \end{cases} \quad (1)$$

The cost of the optimal alignment is then given by $D(m, n)$. The calculation of DTW then is to find the minimum cost warping path in the cumulative matrix, which is made with a memory and time complexity of $O(m * n)$.

Determining a correlation or alignment between Q and C is to find the shortest warping path from the point $(1, 1)$ to point (m, n) . If Q and C are similar, warping path is on the diagonal. However, if the two sequences are different the warping path is far from the diagonal. Given the huge number of possible warping paths it can be more efficient to find the shortest path with some restrictions shown in the next section.

III. DTW CONSTRAINTS

Dynamic Time Warping method is to select, among all the physically possible paths, wherein one with the lowest total distance $DTW(Q, C)$ and which is the shortest path. The number of possible warping paths through the grid is exponentially explosive. So the enumeration of all warping paths is time consuming. It is necessary to consider techniques to reduce the computational effort [11]. In this section we will present different local and global constraints for speeding up DTW. We define the warp path by $W = w_1, w_2, \dots, w_k, \dots, w_K (\max(m, n) \leq K \prec m + n - 1)$.

A. Local constraints

- Boundary condition: the warping path starts with the point $(1, 1)$ corresponding to the top of the two sequences and ends on (m, n) corresponding to the end of the two sequences.
- Continuity condition: it inhibits the displacement for more than one step at a time.
- Monotonicity condition: the warping path does not turn on itself, the indices i and j either remain the same or increase, they never decrease.

B. Global constraints

1) *Sakoe Chiba band and Itakura parallelogram*: Generally, it may that DTW does not give the correct alignment between two sequences. It will try to find the best warping path based on the minimum distance. It can generate an undesirable warping path. A good alignment path is unlikely to be too far from the diagonal. A window or band of size r is defined above and to the right of the diagonal which limits the number of cells accessible in the cumulative matrix. Figure (1) shows the most used global constraints Sakoe Chiba band [7] and Itakura parallelogram [12]. Sakoe Chiba window (Figure (1)(a)) is a band of fixed width $(2r + 1)$ around the diagonal. This constraint is equivalent to only fill the matrix for the terms of indices (i, j) verifying the equation (2):

$$|i - j| \in [0, r] \quad (2)$$

For Itakura parallelogram (Figure (1)(b)), it is to restrict the

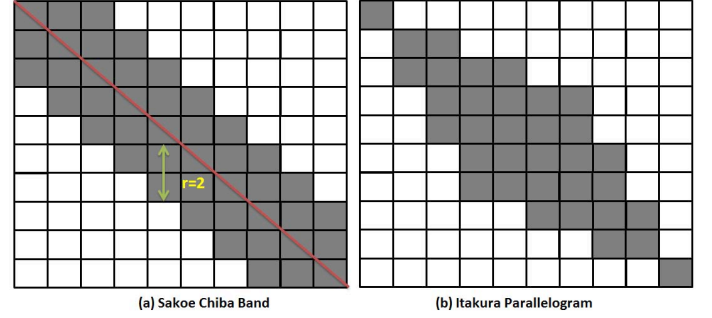


Fig. 1. Global constraints

paths in a parallelogram with one of the diagonals is the diagonal path. The application of the previous two constraints has a positive effect on the temporal complexity. It does not consider the calculation of the terms away from the diagonal.

2) *Slope constraint*: Eligible warping paths can be limited by the restriction of the slope, which prevents excessive movement in one direction. This constraint can be expressed as a/b where a is the number of steps along the x axis and b the number of steps along the y axis. This constraint forces the alignment along the diagonal and prevents very short parts of the sequence to be aligned with very long parts [13].

C. DTW algorithm

In this section we will present how to fill the cumulative matrix row by row and left to right (in this way, we are sure, when one considers the item (i, j) , the terms of indices $(i-1, j)$, $(i, j-1)$ and $(i-1, j-1)$ required to calculate the current term are pre-calculated).

In brief the DTW algorithm consists of three steps:

- Calculation of local distances.
- Dynamic programming for the global optimum.
 - Calculation of distances increased. Keep only the minimum predecessors of each point (principle of optimality) $\Rightarrow$ local optimum.
 - Backtrack to find the minimum distance $\Rightarrow$ global optimum.
- Result: warping path which consists of the alignment of the two time series.

An alternate implementation can be used that does not require the entire cost matrix to be filled and only needs 2 columns to be stored at any one time: current and last columns. A very interesting feature is that DTW is symmetric ($DTW(Q, C) = DTW(C, Q)$). So switching the two time series parameters does not affect the final warp cost but can reduce the space complexity by allowing m to be set as the shorter time series and only requiring 2 lines of size m rather than 2 larger lines of size n . The space complexity is $2 * m$ (if $m \leq n$), $2 * n$ otherwise.

Find the Warp Path by searching the matrix from the solution at (m, n) to the beginning at $(0, 0)$. At each step move through the matrix 1 step left, down, or diagonal, whichever has the smallest cost. Favor diagonal moves and moves towards the $i == j$ axis to break ties.

IV. DERIVATIVE DYNAMIC TIME WARPING

According to this principle, DTW tends to explain variations in the Y axis, deforming the X axis. However, this may lead to undesirable alignments. To address this problem (Keogh and Pazzani) [6] proposed in 2001 an amendment to the classic DTW called "Derivative Dynamic Time Warping (DDTW)". This method takes into account the shape of time series and the first derivative of sequences. The first term in the calculation of the cumulative distance is not the euclidean distance. It is the derivative of the two time series calculated according to the equation (3):

$$D(X) = \frac{(X_i - X_{i-1}) + (X_{i+1} - X_{i-1})/2}{2} \quad (3)$$

This algorithm tries to avoid the problem of singularity caused by DTW alignment. DDTW minimizes the number of duplicate points like in Figure (2).

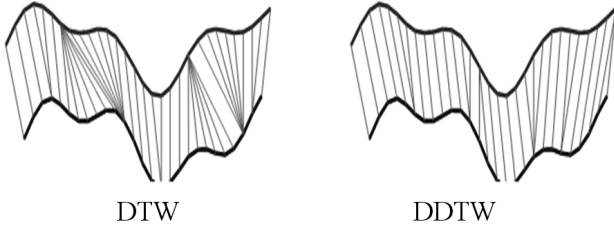


Fig. 2. Comparison between DTW and DDTW [6]

V. DYNAMIC WARPING WINDOW

In this section we will present a new approach for calculating DTW based on the upper bound. We begin by citing the different upper bounds defined in the literature then we will describe the proposed.

A. Upper bounds for DTW

One way to upper bound DTW is to calculate the Manhattan distance between the two sequences. Also DTW can be upper bounded by the euclidian distance. (R. Tavenard, 2011) [14] defines the following upper bounds:

1) *UB_Keogh upper bound*: LB_Keogh is calculated by approximating one of the two sequences Q and C , by its envelope defined in [15]. It is to estimate the similarity between two sequences by the sum of distances between each of the elements of C and the closest point in the range $[l_i, u_i]$ as defined in [15]. UB_Keogh use the same principle, by summing the distances between these elements and the farthest point in $[l_i, u_i]$. It is defined by:

$$UB_Keogh(Q, C) = \sum_{i=1}^n \max(u_i - c_i, c_i - l_i) \quad (4)$$

This upper bound has the same computational complexity as LB_Keogh [15].

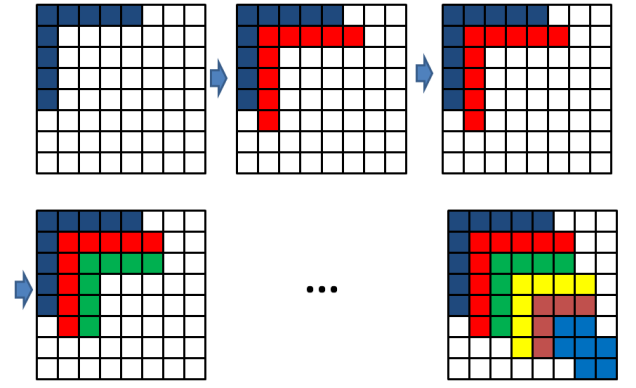


Fig. 3. Finding the dynamic warping window

2) *UB_Yi upper bound*: This upper bound is calculated as given in equation (5)

$$UB_Yi(Q, C) = \begin{cases} (1) \min(\sum_i q_i - \min(C), \sum_i \max(Q) - c_i) \\ (2) \sum_i \max(q_i - \min(C), \max(C) - q_i) \end{cases} \quad (5)$$

- (1): $\min(Q) \geq \min(C)$
- (2): $\min(Q) < \min(C)$

B. Dynamic Warping Window algorithm description

The calculated upper bound is used to determine the number of cells evaluated in the cumulative matrix. We will construct a window of dynamic size. When we find a value of cell above the upper bound, then stop filling the current column. Table (I) shows an example of filling the cumulative matrix. Once the value of a cell is above the upper bound, the following cells of the current column are ignored. This method gives the exact distance value given by DTW. It accelerates the functioning of DTW algorithm. We can use not the entire cost matrix to be filled and we need only 2 columns to be stored at any one time: current and last columns. Then space complexity is $2 * m$ where m the length of time series. For this first variant, denoted by DWW1 (Figure (4 (a))); only cells below the diagonal are evaluated. Cells with a value greater than the upper bound are ignored. For the second variant (DWW2) (Figure (4 (b))) we ignore also cells symmetric to those eliminated by DWW1 (Table (II)). Figure (3) shows an example of finding dynamically the warping window.

To determine the efficiency of our method, the CPU time for the KNN classification is calculated.

C. Dynamic Warping Window Complexity

DWW reduce the temporal and spatial complexity by minimizing the number of calculated cells in the cumulative matrix. The number of cells not calculated is determined dynamically during the process of filling the cumulative matrix. This means that the size of the warping window is not defined a priori. It is to prove that DWW has a linear time complexity. In fact, as shown in [7] if the size of the Sakoe chiba band has a fixed size then the CPU time complexity is linear. Thus as shown in Figure (5), the warping window determined dynamically by

TABLE I
DWW1

	-0.46	-0.62	-0.71	-0.58	-0.17	0.77	1.97	0.24	0.99	1.73
-0.87	0.1681	0.3706	0.515	0.6446	0.6975	6.3619	9.8588	15.2877	16.3078	22.5079
-0.91	0.2306	0.2522	0.3006	0.3406	0.4927	6.9443	10.4828	16.0589	16.6566	23.2303
-0.84	0.2562	0.2706	0.2691	0.2812	0.5116	7.4096	11.4387	17.1392	17.6465	24.1642
-0.82	0.3403	0.3651	0.3367	0.3267	0.4037	6.6537	10.6138	16.6163	17.8932	24.4586
-0.23	0.8303	0.8879	0.7856	0.7492	0.3303	4.6984	7.1948	11.3564	11.8748	16.7148
1.92	3.5199	3.6527	3.3777	3.2773	1.3303	1.6528	2.0624	3.2724	3.3208	4.9084
1.41	11.5855	11.8143	11.2738	11.0614	6.1703	1.3328	1.6464	1.6564	3.6728	3.3244
1.87	-	-	-	-	-	4.1552	2.7017	4.3033	1.7525	4.9566
0.55	-	-	-	-	-	5.0201	2.8781	3.4761	1.9461	2.8341
2.03	-	-	-	-	-	-	2.9805	2.8977	3.3385	2.0361

TABLE II
DWW2

	-0.46	-0.62	-0.71	-0.58	-0.17	0.77	1.97	0.24	0.99	1.73
-0.87	0.1681	0.3706	0.515	0.6446	0.6975	6.3619	9.8588	-	-	-
-0.91	0.2306	0.2522	0.3006	0.3406	0.4927	6.9443	10.4828	-	-	-
-0.84	0.2562	0.2706	0.2691	0.2812	0.5116	7.4096	11.4387	-	-	-
-0.82	0.3403	0.3651	0.3367	0.3267	0.4037	6.6537	10.6138	-	-	-
-0.23	0.8303	0.8879	0.7856	0.7492	0.3303	4.6984	7.1948	-	-	-
1.92	3.5199	3.6527	3.3777	3.2773	1.3303	1.6528	2.0624	3.2724	3.3208	-
1.41	11.5855	11.8143	11.2738	11.0614	6.1703	1.3328	1.6464	1.6564	3.6728	3.3244
1.87	-	-	-	-	-	4.1552	2.7017	4.3033	1.7525	4.9566
0.55	-	-	-	-	-	5.0201	2.8781	3.4761	1.9461	2.8341
2.03	-	-	-	-	-	-	2.9805	2.8977	3.3385	2.0361

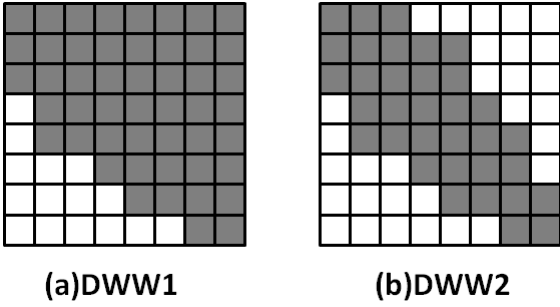


Fig. 4. Dynamic Warping Window

DWW2 is included in a Sakoe Chiba band of fixed radius r equal to the higher distance between a point in the extremity of the warping window and the diagonal path ($r = 13$ in the picture).

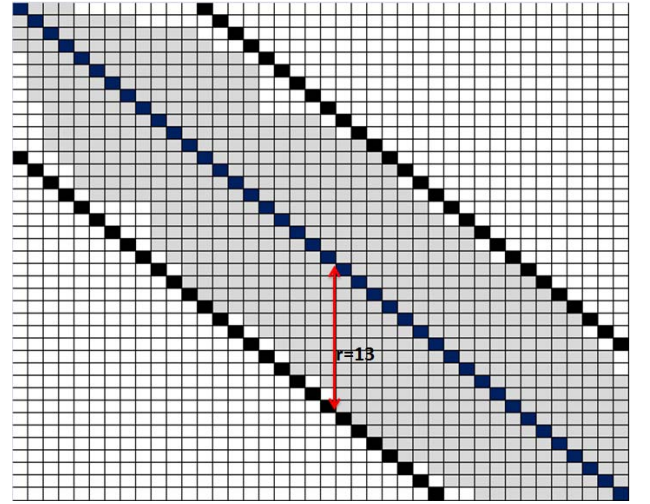


Fig. 5. Dynamic Warping Window Complexity

VI. EXPERIMENTAL RESULTS

In this section we present the approximation error rate of the DWW1 and DWW2 in order to measure the accuracy of our method. In order to prove the efficiency of the new approach we present the KNN classification results for some instances presented in [16]. The used upper bound is the euclidean distance.

A. Dynamic Warping Window accuracy

The accuracy of the DWW can be measured using the equation (6).

$$\text{approxErrorRate} = \frac{\text{approxDist} - \text{DtwDist}}{\text{DtwDist}} * 100 \quad (6)$$

As shown in the table (III), the accuracy obtained via DWW1 is always 100% for all tested data sets. The approximation error of DWW2 is very close to 0% for all tested instances.

B. Classification results

To promote our new approach to DTW and its variants, we compute the CPU computational effort for KNN classification using DTW, DDTW, DWW1 and DWW2. UCR Time Series data sets used in this paper are taken from the UCR Time Series Classification/Clustering Page [16]. The UCR Time Series data repository covers various domains. The time series length ranges from 60 (Synthetic Control data set) to 1882 (InlineSkate data set). The number of classes in a data

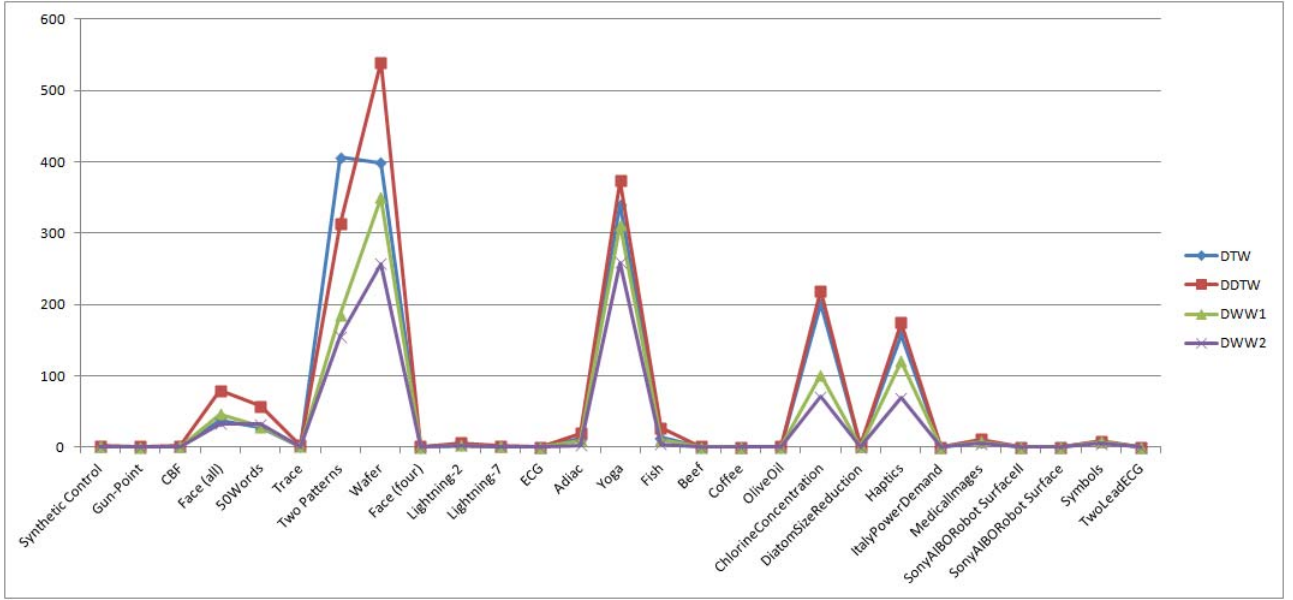


Fig. 6. CPU time computation using KNN classification

TABLE III
APPROXIMATION ERROR RATES

Data set	DWW1 (%)	DWW2 (%)
synthetic_control	0.0	0.0
Gun-Point	0.0	1.248232752
CBF	0.0	0.0
Face (all)	0.0	0.0
50Words	0.0	0.399680595
Trace	0.0	5.20172565
Two_Patterns	0.0	0.95946341
Wafer	0.0	0.023306616
Face (four)	0.0	0.0
Lightning-2	0.0	9.00E-04
Lightning-7	0.0	0.015189196
ECG	0.0	0.602846604
Adiac	0.0	0.0
Beef	0.0	0.96525247
Coffee	0.0	4.13733333
Yoga	0.0	0.0
Fish	0.0	0.013075347
OliveOil	0.0	0.002442091
ChlorineConcentration	0.0	0.0
DiatomSizeReduction	0.0	6.56E-05
Haptics	0.0	0.0
ItalyPowerDemand	0.0	0.51622183
MedicalImages	0.0	0.882327002
SonyAIBORobot SurfaceII	0.0	0.014032138
SonyAIBORobot Surface	0.0	0.366274995
Symbols	0.0	0.12944771
TwoLeadECG	0.0	0.17459945

set ranges from 2 to 50. Each data set is divided into a fixed training set and testing set. The number of samples in a training set ranges from 20 (MoteStrain data set and SonyAIBORobot Surface data set) to 1800 (Non-Invasive Fetal ECG Thorax1 and Non-Invasive Fetal ECG Thorax2 data sets). The number of testing samples ranges from 28 (Coffee data set) to 8236 (StarLightCurves data set).

For our test, 20 data sets are selected. DWW gives a good compromise between computational speed and accuracy of KNN classification.

As shown in Figure (6) the best execution time is given by DWW2. The highest execution time is given by DDTW. For some instances the CPU computation time necessary using DWT is two times greater than DWW2 (ChlorineConcentration, DiatomSizeReduction etc).

K-NN classification results are given in the Table (IV). As shown in this table, DWW1 preserves the same classification results as given by DTW. This can be explained by that DWW1 gives the same distance value as DTW (Table (III)). For DDTW, it improves the classification results for four instances (50Words, ECG, Fish and ChlorineConcentration) and degrades the results for almost all data sets. DWW2 improves the classification results for six instances (Face (all), 50Words, Lightning-7, Beef, ItalyPowerDemand and TwoLeadECG), it degrades results for one instance (MedicalImages) and it preserves the same results given by DTW for the rest of instances.

VII. CONCLUSION AND FUTURE WORKS

The results given by DWW1 are the same as given by DTW. This method manages to speedup DTW while avoiding the degradation of results. The variant DWW2 speeds up the operation of DTW while giving a close approximation of the value of DTW. Instead Derivative Dynamic Time Warping gives less accurate results and higher execution time. We aim to try other variants of DTW such as FastDTW. We find that the use of DTW with KNN gave bad results for some instances such as "Haptics". To improve the classification results we are intended to adapt the DTW distance to the used field using distance learning. The idea is to learn parameters of DTW and other variants using as example the Large Margin Nearest Neighbors algorithm.

TABLE IV
KNN CLASSIFICATION ERROR RATES

Data sets	DTW	DDTW	DWW1	DWW2
Synthetic Control	0.007	0.433	0.007	0.007
Gun-Point	0.093	0.0067	0.093	0.093
CBF	0.003	0.4078	0.003	0.003
Face (all)	0.192	0.127	0.192	0.192
50Words	0.3099	0.3077	0.3099	0.3099
Trace	0.0	0.0	0.0	0.0
Two Patterns	0.0	0.0	0.0	0.0
Wafer	0.020	0.022	0.020	0.020
Face (four)	0.17	0.375	0.17	0.17
Lightning-2	0.131	0.328	0.131	0.131
Lightning-7	0.274	0.425	0.274	0.247
ECG	0.23	0.17	0.23	0.23
Adiac	0.396	0.414	0.396	0.396
Yoga	0.164	0.18	0.164	0.164
Fish	0.1657	0.1029	0.1657	0.1657
Beef	0.5	0.4667	0.5	0.4667
Coffee	0.179	0.179	0.179	0.179
OliveOil	0.133	0.2	0.133	0.133
ChlorineConcentration	0.352	0.317	0.352	0.352
DiatomSizeReduction	0.033	0.065	0.033	0.033
Haptics	0.623	0.714	0.623	0.623
ItalyPowerDemand	0.05	0.086	0.05	0.0496
MedicalImages	0.263	0.337	0.263	0.2697
SonyAIBORobot SurfaceII	0.169	0.142	0.169	0.169
SonyAIBORobot Surface	0.275	0.2	0.275	0.275
Symbols	0.05	0.03	0.05	0.05
TwoLeadECG	0.096	0.0053	0.096	0.094

REFERENCES

- [1] A. Efrat, S. Venkatasubramanian, and Q. Fan, "Curve matching, time warping, and light fields: New algorithms for computing similarity between curves," *J. Mathematic Imaging and Vision*, vol. 2007.
- [2] J. Zhang and B. Qin, "Dtw speech recognition algorithm of optimization template matching," in *World Automation Congress (WAC), 2012*, June 2012, pp. 1–4.
- [3] A. Kuzmanic and V. Zanchi, "Hand shape classification using dtw and lcss as similarity measures for vision-based gesture recognition system," in *EUROCON, 2007. The International Conference on #34;Computer as a Tool #34;*, 2007, pp. 264–269.
- [4] S. Hussain and A. Rashid, "User independent hand gesture recognition by accelerated dtw," in *Informatics, Electronics Vision (ICIEV), 2012 International Conference on*, May 2012, pp. 1033–1037.
- [5] V. Niennattrakul and C. Ratanamahatana, "On clustering multimedia time series data using k-means and dynamic time warping," in *Multimedia and Ubiquitous Engineering, 2007. MUE '07. International Conference on*, 2007, pp. 733–738.
- [6] E. J. Keogh and M. J. Pazzani, "Derivative dynamic time warping," in *First SIAM International Conference on Data Mining (SDM2001)*, 2001.
- [7] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *Acoustics, Speech and Signal Processing, IEEE Transactions on*, vol. 26, no. 1, pp. 43 – 49, feb 1978.
- [8] D. J. Berndt and J. Clifford, "Using Dynamic Time Warping to Find Patterns in Time Series," in *KDD Workshop*, 1994, pp. 359–370.
- [9] E. J. Keogh and M. J. Pazzani, "Scaling up dynamic time warping for datamining applications," in *KDD*, R. Ramakrishnan, S. J. Stolfo, R. J. Bayardo, and I. Parsa, Eds. ACM, 2000, pp. 285–289.
- [10] R. Bellman, *Dynamic Programming*, 1st ed. Princeton, NJ, USA: Princeton University Press, 1957.
- [11] "Dynamic time warping," vol. 17, 2007.
- [12] F. Itakura, "Minimum prediction residual principle applied to speech recognition," *Acoustics, Speech and Signal Processing, IEEE Transactions on*, vol. 23, no. 1, pp. 67–72, 1975.
- [13] C. Ann and R. E. Keogh, "Making time-series classification more accurate using learned constraints."
- [14] R. Tavenard, "Indexation de squences de descripteurs," Ph.D. dissertation, 2011.
- [15] E. Keogh, "Exact indexing of dynamic time warping," 2002.
- [16] Z. Q. H. B. H. Y. X. W. L. . R. Keogh, E., "The ucr time series classification/clustering homepage: www.cs.ucr.edu/~eamonn/time_series_data/," 2011.